

maquette-gltf

Render glTF 2.0 assets in Typst

Source	github.com/bernsteining/maquette
Typst Universe	typst.app/universe/package/maquette-gltf
Live demo	bernsteining.github.io/maquette/?model=tokyo.glb

Version 0.1.0 · October 26, 2026

CONTENTS

Introduction . . . 3

Quickstart . . . 3

Config reference . . . 4

Loading assets . . . 5

Camera . . . 5

- Orbit . . . 5
- Zoom and lens . . . 6
- Explicit camera . . . 6

Lighting . . . 7

- Key light and ambient . . . 7
- Image-based lighting . . . 7

Shadows and ground . . . 8

Post-processing . . . 8

- Ambient occlusion . . . 8
- Tone mapping and exposure . . . 9
- Anti-aliasing . . . 9

Animation . . . 10

Textures . . . 11

Output . . . 11

Metadata . . . 12

Model credits . . . 12

Introduction

maquette-gltf renders glTF 2.0 files (`.glb`, `.gltf`) as images in your Typst document, with nothing to install.

> **Try it live** https://bernsteining.github.io/maquette/?model=tokyo.glb
>
> Load a glTF, orbit it, play its animation, adjust every option in this manual, and copy the Typst code.

It provides two functions:

- **`render-gltf(model, ..options)`** renders the model as an image.
- **`get-gltf-info(model)`** returns its size, triangle count and animation length.

The examples use two models, the Fox and the Littlest Tokyo credits are at the end.

Quickstart

Read the model as bytes and pass it to `render-gltf` (left). The examples below reuse two settings: `studio`, a lighting look that fills the frame (right), and `side`, a wide side view for the Fox (below).



```
#import "@preview/maquette-gltf:0.1.0": render-gltf

#let tokyo = read("tokyo.glb", encoding: none)
#render-gltf(tokyo)
```



```
#let studio = (background: none, ssao: true,
  ibl: (intensity: 1.2), zoom: 1.5)
#render-gltf(tokyo, ..studio)
```



```
#let side = (width: 600, height: 300,
  azimuth: 90, elevation: 8)
#render-gltf(fox, side, ..studio,
  zoom: 2.3, animation-index: 2)
```

Config reference

All optional; defaults shown.

```
{ // ── Viewport ──────────────────────────────────────────────
  "width": 500,                    // Output width in pixels
  "height": 500,                   // Output height in pixels
  "background": "#f0f0f0",         // Hex colour; none, "none" or "" = transparent
  // ── Camera ────────────────────────────────────────────────
  "camera": null,                  // Eye position (x, y, z); null = orbit around the model
  "center": [0, 0, 0],             // Look-at target (overridden by auto-center)
  "up": [0, 1, 0],                 // Up direction (glTF is Y-up)
  "azimuth": 30,                   // Orbit horizontal angle, degrees
  "elevation": 20,                 // Orbit vertical angle, degrees
  "distance": null,                // Orbit distance; null = fit the model
  "zoom": 1.0,                     // Move closer than the automatic framing
  "fov": 45,                       // Vertical field of view, degrees
  "auto-center": true,             // Aim at the scene's bounding-box centre
  "auto-fit": true,                // Frame the scene to fill the image
  "camera-auto-use": true,         // Use the asset's first camera when it has one
  "camera-index": null,            // Use an authored camera by index
  "camera-name": null,             // Use an authored camera by name
  "scene-index": null,             // Scene to render; null = the default scene
  // ── Lighting ──────────────────────────────────────────────
  "light-dir": [1, 2, 3],          // Key light direction when the asset has no lights
  "ambient": 0.2,                  // Constant ambient (0-1); unused with ibl
  "ibl": null,                     // true, or {intensity, sky, ground, rotation, hdr}
  "shadows": null,                 // true, or {resolution, softness, light-size, bias, normal-bias, slope-bias}
  "ground": null,                  // true, or {color, roughness, size-scale, y}
  "cull-backface": true,           // Skip back faces of single-sided materials
  // ── Post-processing ───────────────────────────────────────
  "ssao": null,                    // true, or {samples, radius, bias, strength, space}
  "antialias": 4,                  // 0 off, 1 FXAA, 2-3 SSAA x2/x4, 4-5 FXAA + SSAA x2/x4
  "tone-mapping": "",              // "", "reinhard" or "aces"
  "exposure": 1.0,                 // Light multiplier before tone mapping
  // ── Animation and materials ───────────────────────────────
  "time": 0.0,                     // Animation time, seconds
  "animation-index": null,         // Animation to use (from 0); null = all together
  "material-variant": 0,           // KHR_materials_variants index
  // ── Textures ──────────────────────────────────────────────
  "texture-max-size": null,        // Downsample textures to this side, pixels
  "no-textures": false             // Skip textures (fast drafts)
}
```

Loading assets

- **`.glb` files**: read with `encoding: none` and pass the bytes, as above.
- **Split `.gltf` files**: pass the path and a `read:` function, which loads the `.bin` and texture files next to it.

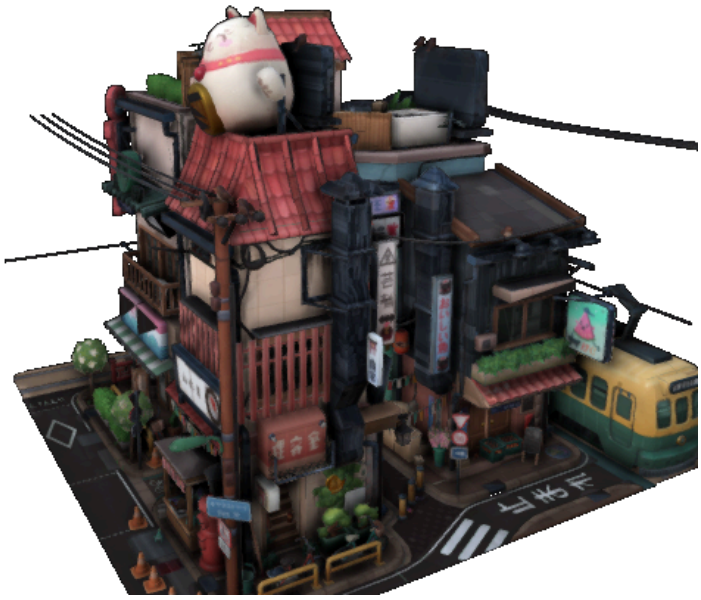


```
#render-gltf("gltf/fox/Fox.gltf",
  read: p => read(p, encoding: none),
  side, ..studio, zoom: 2.3)
```

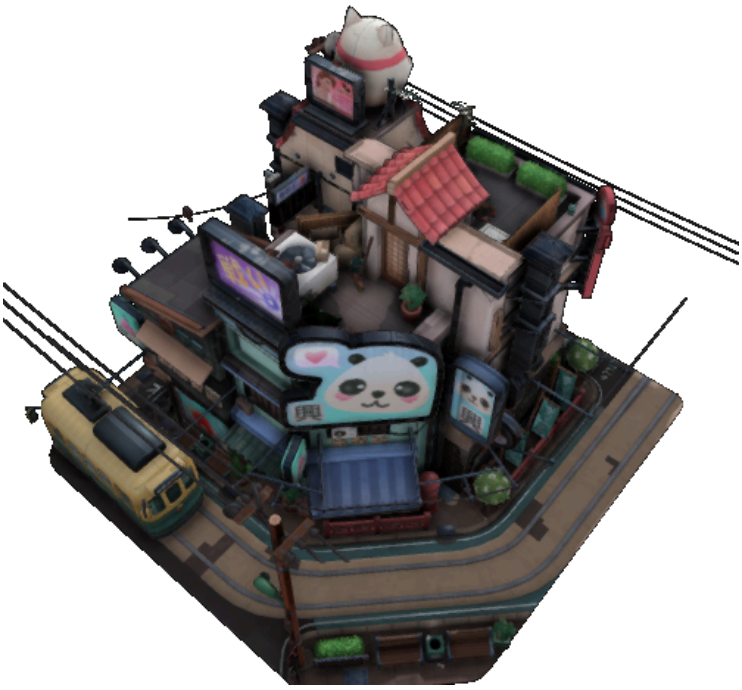
Camera

Orbit

`azimuth` turns the camera around the model and `elevation` raises it, in degrees.



```
#render-gltf(tokyo, ..studio,
  azimuth: 120, elevation: 15)
```



```
#render-gltf(tokyo, ..studio,
  azimuth: -40, elevation: 50)
```

Zoom and lens

`zoom` moves closer than the automatic framing. `fov` sets the lens angle in degrees: narrow flattens perspective, wide exaggerates it.



```
#render-gltf(tokyo, ..studio,
  zoom: 2.6)
```



```
#render-gltf(tokyo, ..studio,
  fov: 100, zoom: 2.2)
```

Explicit camera

`camera`, `center` and `up` place the camera in scene coordinates; turn off `auto-center` and `auto-fit` to use them as given. Models with their own cameras use the first one; pick another with `camera-index`, or ignore them with `camera-auto-use: false`.

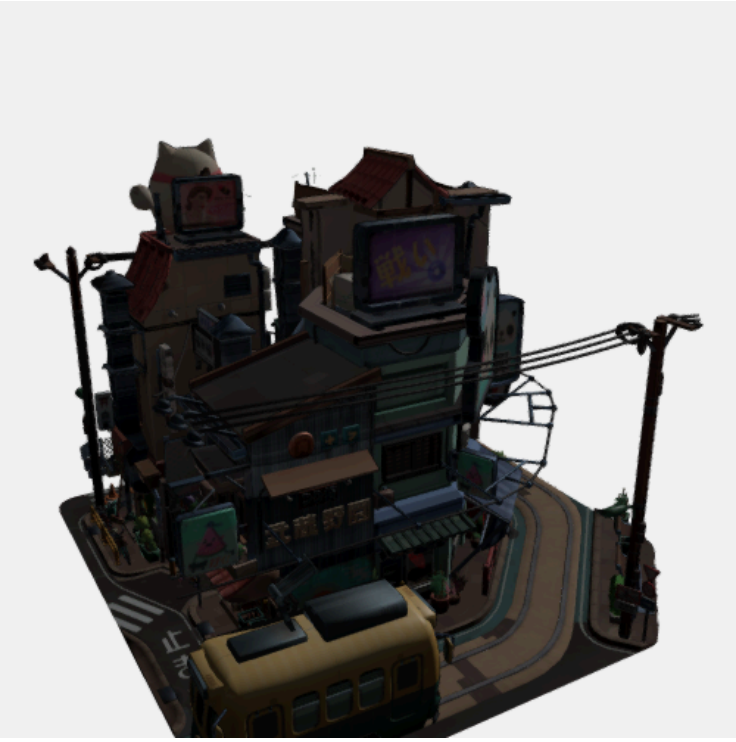


```
#render-gltf(tokyo, (width: 600, height: 280), ..studio,
  camera: (-60, 120, 640),
  center: (-86, 5, -25),
  up: (0, 0.989, -0.145),
  auto-center: false, auto-fit: false)
```

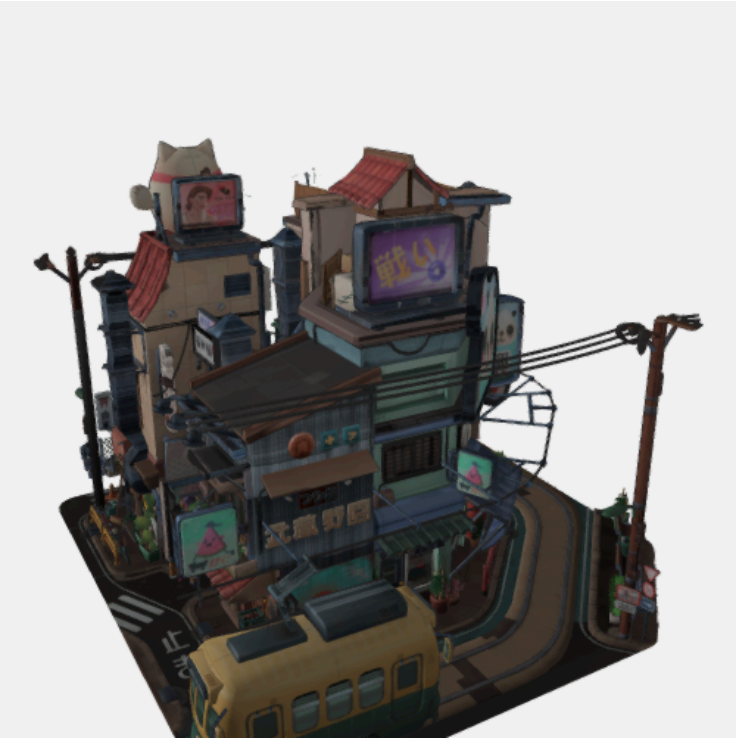
Lighting

Key light and ambient

Without `ibl`, `light-dir` points the main light and `ambient` adds a flat fill from 0 to 1.



```
#render-gltf(tokyo, zoom: 1.5,
  light-dir: (-3, 2, 1), ambient: 0.1)
```



```
#render-gltf(tokyo, zoom: 1.5,
  light-dir: (3, 1, 2), ambient: 0.6)
```

Image-based lighting

`ibl` lights the model from a surrounding environment, which suits physically based materials.

- `intensity`, `sky` and `ground` colours, and `rotation` (radians) tune the built-in sky.
- `hdr: read("studio.hdr", encoding: none)` uses a real environment map, such as the free ones on Poly Haven.



```
#render-gltf(tokyo, ..studio,
  ibl: (intensity: 0.7))
```



```
#render-gltf(tokyo, ..studio,
  ibl: (sky: "#ffb070", ground: "#203050",
    rotation: 2, intensity: 1.4))
```

Shadows and ground

- **`shadows`**: `true`, or set `softness`, `resolution` (default 512) and `light-size`.
- **`ground`**: a floor that catches them; set its `color`, `roughness` and `size-scale`.



```
#render-gltf(fox, side, ..studio, zoom: 2,
  elevation: 30, azimuth: 70,
  shadows: (softness: 3),
  ground: (color: "#c9ced8", size-scale: 8))
```

Post-processing

Ambient occlusion

`ssao` darkens creases and contact areas, adding depth; `strength` sets how dark.



```
#render-gltf(tokyo, ..studio,
  ssao: false)
```



```
#render-gltf(tokyo, ..studio,
  ssao: (strength: 2.5))
```

Tone mapping and exposure

Bright light clips to white. `tone-mapping` (`"reinhard"` or `"aces"`) compresses it smoothly; `exposure` brightens or darkens.



```
#render-gltf(tokyo, ..studio,
  ibl: (intensity: 3), tone-mapping: "")
```



```
#render-gltf(tokyo, ..studio,
  ibl: (intensity: 3), tone-mapping: "aces",
  exposure: 1.2)
```

Anti-aliasing

`antialias` smooths edges, from 0 (off) to 5 (best); the default is 4.



```
#render-gltf(fox, side, ..studio,
  antialias: 0, zoom: 5)
```



```
#render-gltf(fox, side, ..studio,
  antialias: 5, zoom: 5)
```

Animation

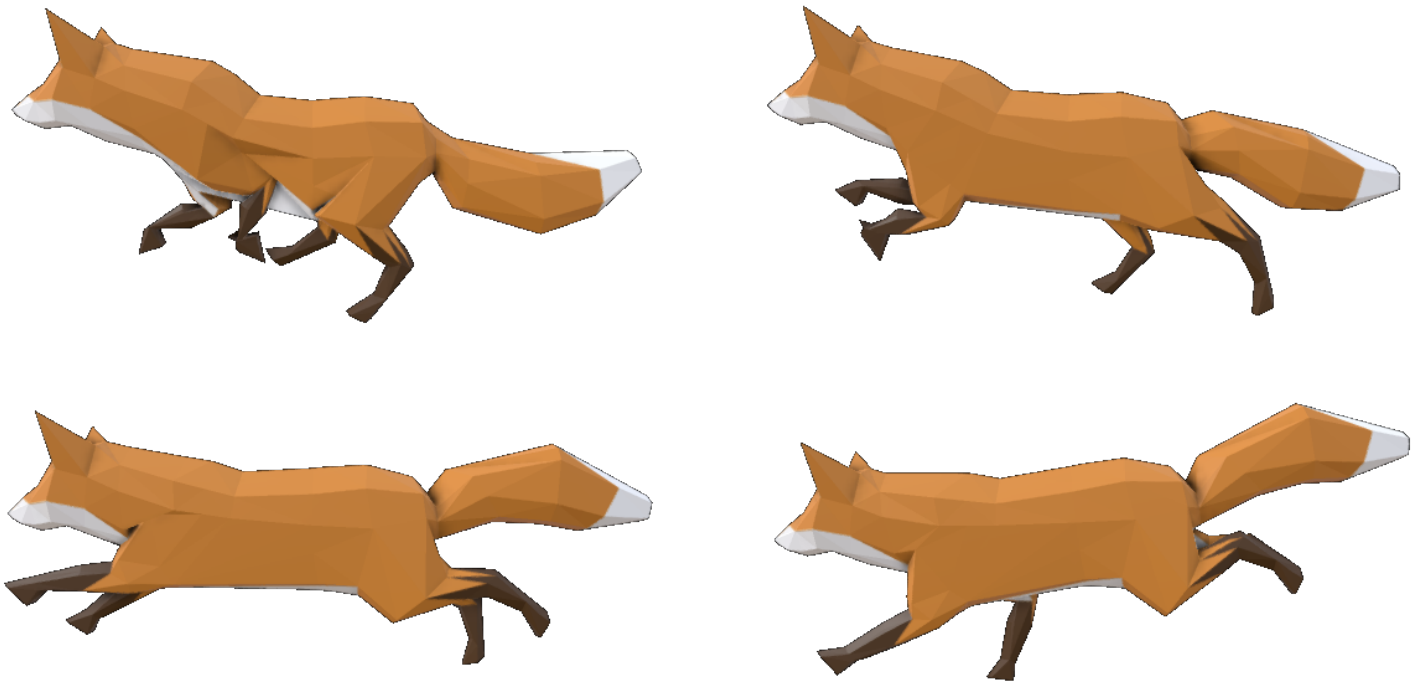
A glTF file can store animations: the Fox's file holds three, the Fox looking around, walking and running. A render is a still image, so two options choose what it shows:

- **`animation-index`**: which animation to use, counted from 0. Without it, all of the file's animations are applied together.
- **`time`**: which moment of that animation to show, in seconds from its start.



```
#render-gltf(fox, side, ..studio, zoom: 2.3,
  animation-index: 1, time: 0.4)
```

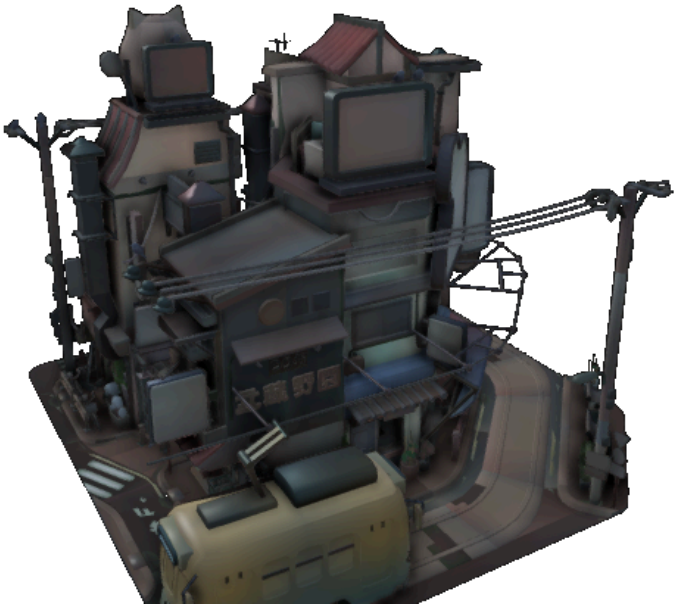
Stepping `time` gives the frames of the run:



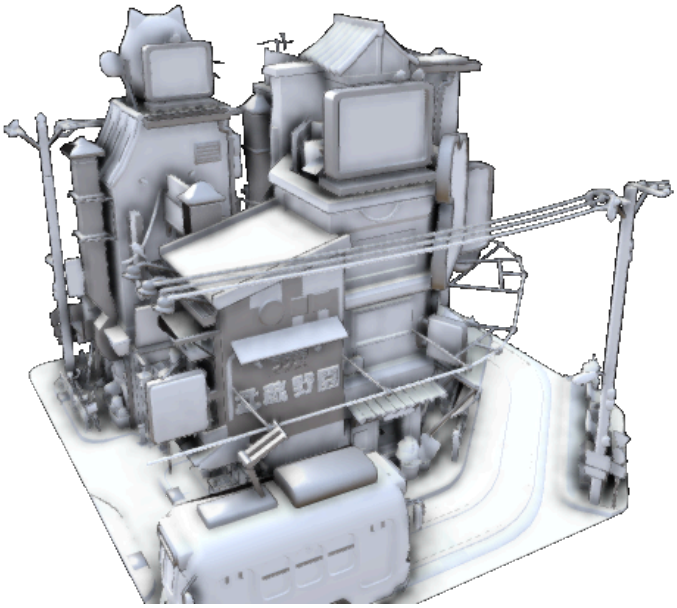
```
#grid(columns: 2, gutter: 0.6em, ..range(4).map(i =>
  render-gltf(fox, side, ..studio, zoom: 2.3,
    animation-index: 2, time: i * 0.18)))
```

Textures

For faster drafts, `texture-max-size` shrinks textures and `no-textures: true` skips them.



```
#render-gltf(tokyo, ..studio,
  texture-max-size: 16)
```



```
#render-gltf(tokyo, ..studio,
  no-textures: true)
```

Output

`width` and `height` in a dictionary set the image size in pixels; as named arguments they set its size on the page. `background` sets a colour, or `none` for transparent.



```
#render-gltf(tokyo, ..studio, zoom: 2.4,
  (width: 600, height: 300), width: 100%)
```



```
#render-gltf(tokyo, ..studio,
  background: "#16181f")
```

Metadata

`get-gltf-info` returns the model's metrics without rendering it.

triangles	576
bbox_min	−14.6, −1.3, −91.1
bbox_max	14.6, 74.5, 72.1
center	0, 36.6, −9.5
radius	91.2
max_animation_time	3.4

```
#let info = get-gltf-info(fox)
#let fmt(v) = if type(v) == array {
  v.map(fmt).join(", ")
} else { str(calc.round(v, digits: 1)) }
#table(columns: 2, ..info.pairs()
  .map(((k, v)) => (raw(k), fmt(v)))
  .flatten())
```

Model credits

Model	Authors	Licence
Fox	PixelMannen (model), tomkranis (rigging and animation), AsoboStudio and scurest (glTF)	CC0 1.0, CC BY 4.0
Littlest Tokyo	Glen Fox	CC BY 4.0